

Last Time:

- Dual Spin / Momentum Bias
- Flat spin recovery
- Actuator Jacobian

Today:

- Attitude Regulation (Pointing)
- Slew Maneuvers

Attitude Regulation:

- Problem: hold a fixed attitude with a feedback controller reaction wheels (or thrusters).

* Full gyrostatt dynamics:

$$\dot{q} = \underbrace{\frac{1}{2} G(q)}_{\text{Attitude Jacobian}} \omega \quad (\text{or } \dot{Q} = Q \hat{\omega}) \quad \} \text{ "Kinematics"}$$

$$\dot{\omega} = -J^{-1}(\omega \times (J\omega + p) + \dot{p} - \tau) \quad \} \text{ "Dynamics"}$$

$$\dot{p} = \underbrace{B_w}_{\text{Wheel Jacobian}} u$$

* State is $x = \begin{bmatrix} q \\ \omega \\ p \end{bmatrix}$

- Approach: Linearize about a desired attitude q_d and $\omega = p = 0$

$$x_d = \begin{bmatrix} q_d \\ 0 \\ 0 \end{bmatrix}$$

$$\Rightarrow \dot{\omega} \approx -J^{-1} \dot{p} + J^{-1} \tau$$

* Attitude error:

$$\overset{\text{actual attitude}}{\overset{N}{Q}^B} = \overset{\text{Desired attitude}}{\overset{N}{Q}^D} \overset{\text{Attitude Error}}{\overset{D}{Q}^B} \Rightarrow \underline{\overset{D}{Q}^B} = \underline{\overset{D}{Q}^N} \overset{N}{Q}^B$$

$$Q_e = Q_d^T Q$$

$\Downarrow$

$$q_e = q_d^+ * q = L(q_d) q$$

analogous to " $e = x - x_d$ " for vectors

* Attitude error dynamics

$$\dot{q}_e = q_d^+ \otimes \dot{q} = \frac{1}{2} \underbrace{L^T(q_d) L(q)}_{L(q_0)} H \omega = \frac{1}{2} G(q_e) \omega$$

- Linearize attitude error dynamics about q_d ($q_e = \begin{bmatrix} \frac{1}{2} \\ 0 \\ 0 \end{bmatrix}$)

$$\Rightarrow \dot{q}_e = \frac{1}{2} \underbrace{L(q_0) H}_{I} \omega \approx \frac{1}{2} H \omega = \begin{bmatrix} 0 \\ \frac{1}{2} \omega \end{bmatrix}$$

approximately true for small errors

- Write as axis-angle ϕ :

$$\begin{aligned} \phi &= 2H^T \log(q_e) \Rightarrow \dot{\phi} \approx \omega \\ &= \widetilde{\log(Q_e)} \end{aligned}$$

- Stack everything together:

$$\underbrace{\begin{bmatrix} \dot{\phi} \\ \dot{\omega} \end{bmatrix}}_{\dot{x}} \approx \underbrace{\begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix}}_A \underbrace{\begin{bmatrix} \phi \\ \omega \end{bmatrix}}_x + \underbrace{\begin{bmatrix} 0 & 0 \\ -J^{-1} & J^{-1} \end{bmatrix}}_B \underbrace{\begin{bmatrix} \dot{p} \\ r \end{bmatrix}}_u$$

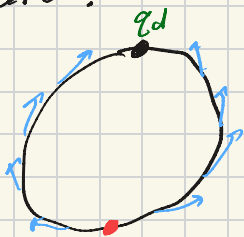
⇒ To 1st order, spacecraft is a double-integrator plant

- Any linear control technique can be used (e.g. PD, LQR)
- Similar to MEKF, we have a step outside the linear controller to compute rotation error + convert to 3-parameter vector:

$$\phi = \log^{\vee}(Q_d^T Q) = \underbrace{2H^T \log(q_d^* * q)}_{\text{can also use vector part of quaternion or Gibbs/Rodrigues vector}} \approx 2H^T L^T(q_d) q$$

can also use vector part of quaternion or Gibbs/Rodrigues vector

- Note: Because of linearization, the controller will have a limited basin of attraction (it won't necessarily perform well or even converge from very large errors).
- There is a general result that no time-invariant feedback law can be globally stabilizing when the configuration manifold is a sphere:



What happens here?

Large-Angle Maneuvers:

- For large-angle maneuvers it's not a good idea to just command the new q_d in the regulator controller.
- We really want a "motion plan" - a nominal trajectory that includes $q(t)$, $\omega(t)$, $p(t)$, $\dot{p}(t)$, $\tau(t)$
- Given a motion plan, we can track it with a (possibly time-varying) linear controller.
- There are many ways to generate one of these trajectories, from closed-form analytic solutions to numerical trajectory optimization.

* Eigen-Axis Slew:

- The rotation from our current attitude Q to our desired attitude Q_d is:

$${}^D Q^B = Q_e = Q_d^T Q$$

- If we want to spin about a fixed axis to get from Q to Q_d , that axis can be found using eigendecomposition of Q_e .
- Rotation axis corresponds to eigenvector with $\lambda = 1$
- While this is the classic method in the literature we can also use the matrix/quaternion log:

$$\phi = \sqrt{\log(Q_e)} = Z H^T \log(q_e)$$

- We can use this axis-angle to create a trajectory:

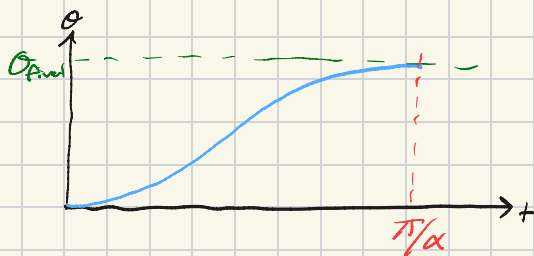
$$\phi(t) = \underbrace{\theta(t)}_{\text{angle as a function of time}} \mathbf{r} \quad \leftarrow \text{unit-vector axis of rotation}$$

$$\Rightarrow \mathbf{Q}(t) = e^{\hat{\phi}(t)}, \quad \dot{\mathbf{Q}}(t) = \dot{\theta}(t) \mathbf{r}$$

- A common choice for $\theta(t)$ is the "versine" function:

$$\theta(t) = \|\theta_{\text{final}}\| \frac{1}{2} (1 - \cos(\alpha t))$$

$\leftarrow$ time constant sets how long the maneuver takes



- Nice because $\dot{\theta}(0) = \dot{\theta}(\pi/\alpha) = 0$ "rest-to-rest"

- Can use any "sigmoid-like" function.

Inverse Dynamics:

- Assume we have a smooth trajectory $\mathbf{q}(t)$ that we can differentiate to get $\dot{\mathbf{q}}(t)$ and $\ddot{\mathbf{q}}(t)$

$$\mathbf{J}\ddot{\mathbf{q}} + \dot{\mathbf{p}} + \boldsymbol{\omega} \times (\mathbf{J}\dot{\mathbf{q}} + \mathbf{p}) = \mathbf{0}$$

$$\Rightarrow \dot{\mathbf{p}} = -\mathbf{J}\ddot{\mathbf{q}} - \boldsymbol{\omega} \times (\mathbf{J}\dot{\mathbf{q}} + \mathbf{p})$$

- Start with $\mathbf{p}(0)$, solve for $\dot{\mathbf{p}}(t)$ and integrate to get $\mathbf{p}(t)$.

- Even easier with thrusters:

$$\tau = J\dot{\omega} + \omega \times J\omega$$

- For a generic mechanical system:

$$\underbrace{M(q)}_{\text{"Mass Matrix"}} \ddot{q} + \underbrace{C(q, \dot{q})}_{\text{"Dynamics Bias" / "Coriolis"}} = \underbrace{B(q)}_{\text{Input Jacobian}} u$$

"Manipulator Equation"

* For us $M = J$, $C(q, \dot{q}) = \omega \times J\omega$

$$\Rightarrow u = B^{-1} [M\ddot{q} + C]$$

Needs to be invertable ("Fully Actuated")

- Inverse dynamics will give the open-loop/feed-forward u(t) controls corresponding to $x(t)$. We still need to close the loop with a feedback controller to compensate model error and reject disturbances